Apuntes Semanales

Diego Araya Cordero
2024087949

03/11/2026

De CFG a Tabla

1. Preproceso
2. First
3. Follow
4. Predict
5. Construir la tabla de parsing



Vamos a ver como hacer el punto 4 (Predict)

Predict

Sea G una CFG

- La función PREDICT[X] regresa un conjunto de terminales y posiblemente $
- El argumento X solo puede ser una regla de gramática
- Antes se deben calcular First y Follow
- X es una regla de la forma A → a

```
if (First (x incluye a ε) )

      PREDICT(X) = (First(x) - {ε}) U Follow(X)

else

      PREDICT(X) = FIRST (x)
```

Predict - Ejemplo

S → [S] S

S → ε

De este ejemplo que ya hemos visto conocemos esto:

- FIRST() = {(, ε}
- FOLLOW() = {$,)}

```
if (First (x incluye a ε) )
     PREDICT(X) = (First(x) - {ε}) U Follow(X)
else
     PREDICT(X) = FIRST (x)
```

Paso a paso:

1. Tomamos el primer elemento de FIRST y lo pasamos por PREDICT ()

   PREDICT(1)

2. No incluye a “ε”, entonces nos vamos por el else

   PREDICT(1) = FIRST (1)

   PREDICT(1) = { (}

3. Vamos con el segundo elemento de FIRST y lo pasamos por PREDICT()

   PREDICT(2)

4. Incluye a “ε”

   PREDICT(2) = (First(2) - {ε}) U Follow()

   PREDICT(2) = {ε, $,)} - {ε}

   PREDICT(2) = {$,)}

Predict - Ejemplo 2

Regla	PREDICT[]
E → T E'	{ (, # }
E' → OP T E'	{ +, - }
E' → ε	{ $,) }
OP → +	{ + }
OP → -	{ - }
T → F T'	{ (, # }
T' → M F T'	{ * }
T' → ε	{ $,), +, - }
M → *	{ * }
F → [E]	{ (}
F → #	{ # }

Ya tenemos los FIRST y FOLLOW

Regla	First[α]
E → T E'	{[, #}
E' → OP T E'	{+, -}
E' → ε	{ε}
OP → +	{+}
OP → -	{-}
T → F T'	{[, #}
T' → M F T'	{*}
T' → ε	{ε}
M → *	{*}
F → [E]	{[}
F → #	{#}

No terminal	FOLLOW
E	{ $,) }
E'	{ $,) }
OP	{ (, # }
T	{ $, +, -,) }
T'	{ $, +, -,) }
M	{ (, # }
F	{ $, +, -, *,) }

De CFG a Tabla

1. Preproceso
2. First
3. Follow
4. Predict
5. Construir la tabla de parsing



Construir la tabla de parsing

- Tabla M:
  - Una fila por cada No terminal
  - Una columna por cada terminal y $

- Se calculan los First[] y Follow[]
- Se calcula el PREDICT de cada regla



Tabla de Parsing - Ejemplo 1

Regla	PREDICT[]
1. S → [S] S	{ (}
2. S → ε	{), $ }



Una fila por cada No terminal
Una columna por cada terminal y $

	[	]	$
S	1	2	2

Número de regla

Tabla de Parsing - Ejemplo 2

1. E → T E'
2. E' → OP T E'
3. E' → ε
4. OP → +
5. OP → -
6. T → F T'
7. T' → M F T'
8. T' → ε
9. M → *
10. F → [E]
11. F → #

	[	#	]	+	-	*	$
E	1	1					
E'			3	2	2		3
OP				4	5		
T	6	6					
T'			8	8	8	7	8
M						9	
F	10	11					

Tabla de Parsing - Ejemplo con las 5 partes

Preproceso - FIRST - FOLLOW

S → IFS | <other>

IFS → if [EX] S EL

EL → else S | ε

EX → O | 1

First()

No term.	Pas 1	Pas 2
S	{<other>}	{if, <other>}
IFS	{if}	
EL	{else, ε}	
EX	{0, 1}	

FOLLOW

No term.	Pas 1	Pas 2
S	{$, else}	
IFS	{$}	{$, else}
EL	{$}	{$, else}
EX	{)}	

Tabla de Parsing - Ejemplo con las 5 partes

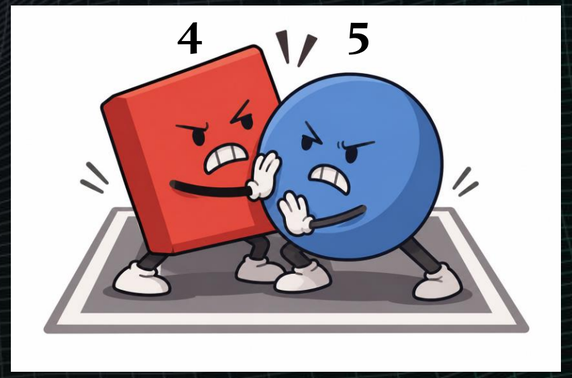
Predict

Regla	First(α)	Predict
1. S → IFS	{if}	{if}
2. S → <other>	{<other>}	{<other>}
3. IFS → if [EX] S EL	{if}	{if}
4. EL → else S	{else}	{else}
5. EL → ε	{else, ε}	{$, else}
6. EX → O	{0}	{0}
7. EX → 1	{1}	{1}

Tabla de Parsing - Ejemplo 4 - Tabla

	if	<other>	else	O	1	$
S	1	2				
IFS	3					
EL			4-5			5
EX				6	7	

Existe un conflicto de reglas
¿¿Qué hacemos en este caso??



Ambigüedad del if-then-else

```
if(0) if(1) other else other
```

2 posibilidades

```
if (0)
    if (1) other
else
    other
```

```
if(0)
    if(1)
        other
    else
        other
```

- Problema del “dangling else”
- Se resuelve forzando que el else se asocie con el if abierto más cercano
- Escoger el 4 sobre el 5

Parsing LL(1)

Es todo lo que hemos estado haciendo

- Tipo de Parsing Top Down
- Es predictivo. No hay backtracking
- Se llaman LL[1] porque:
  - L → “Left to Right”: el texto se procesa de izquierda a derecha
  - L → “Leftmost”: produce derivación Leftmost
  - 1 → Se usa lookahead de 1 token
- Estudiados en 1968 por Lewis y Stearns

Gramáticas LL(1)

- Una CFG es LL(1) si para cualquier par de reglas diferentes $A \to \alpha$ y $A \to \beta$ se cumple que $\text{PREDICT}(A \to \alpha) \cap \text{PREDICT}(A \to \beta) = \emptyset$
- Una gramática es LL(1) si se puede construir una tabla de parsing LL(1) para ella
- Las gramáticas LL(1) son un subconjunto propio de CFG
- No todas son LL(1)

Muchas Gracias